(Klassische) Sicherheitsprinzipien

Dozent: Prof. Dr. Michael Eichberg
Kontakt: michael.eichberg@dhbw.de , Raum 149B
Version: 1.3.0

---

kontrollfragen.de.md.html:
Zur Nach-/Vorbereitung der Vorlesung und Klausur.

Folien: [HTML] https://delors.github.io/sec-klassische-sicherheitsprinzipien/folien.de.md.html
[PDF] https://delors.github.io/sec-klassische-sicherheitsprinzipien/folien.de.md.html.pdf

Fehler melden: https://github.com/Delors/delors.github.io/issues

KI Verwendung: Bei der Erstellung der Unterlagen wurden KI Assistenten (insbesondere Claude aber ggf. auch ChatGPT, Ollama mit Gemma/LLama/Qwen oder OpenCode mit Kimi/Qwen/Deepseek...) unterstützend eingesetzt. Dies erfolgte insbesondere zur Unterstützung bei der Generierung von Grafiken (d. h. SVG Dateien), oder um sich Übersichtstabellen generieren zu lassen. Weiterhin wurde KI zur allgemeinen Qualitätssicherung eingesetzt. Inhalte, die ggf. von der KI vorgeschlagen wurden, wurden im Falle der Übernahme explizit validiert und angepasst.

1. Sicherheitsprinzipien von Jerome Saltzer und Michael Schroeder, 1975 [SaltzerSchroeder75]

Die Sicherheitsprinzipien von Saltzer & Schroeder – Hintergrund

- Saltzer und Schroeder forschten 1975 hauptsächlich im Bereich Betriebssysteme
  Insbesondere Multics. [Saltzer73], [Saltzer74]
- Die Prinzipien sind vielfach direkt aus der Entwicklung von Betriebssystemen abgeleitet

Die acht Sicherheitsprinzipien von Saltzer & Schroeder

***Economy of Mechanism* (auch bekannt unter *Principle of Simplicity*):**

Das Design der Sicherheitsmechanismen sollte so einfach wie möglich sein.

Das Ziel ist es, die Korrektheit des Designs/der Implementierung zu fördern, da dieses dann schneller verstanden wird und auch einfacher getestet werden kann. Weiterhin reduziert es die Angriffsfläche.

(Hintergrund: Der *Happy-Path* ist typischerweise gut getestet und selten ein Sicherheitsproblem, jedoch gibt es in komplexer Software häufig Pfade, die nicht (vollständig) getestet/analysiert wurden und dann ein Problem sind.)

Beispiele:

- das Unix-Dateirechtesystem
- Mailbox-Kommunikation zwischen Systemen und Komponenten
- ...

***Fail-Safe Defaults*:**

Standardmäßig sollte der Zugriff auf Ressourcen (durch Prozesse/Programme) verweigert werden.

Es ist gefordert, dass Zugriffsrechte auf (expliziter) Erlaubnis beruhen und nicht auf (explizitem) Ausschluss. Der gegenteilige Ansatz – „Alles ist erlaubt, solange es nicht explizit verboten ist" – ist nicht tragfähig. Ein fehlendes Recht fällt normalerweise während der Nutzung unmittelbar auf; ein überflüssiges Recht hingegen verursacht keinerlei Probleme und fällt deswegen im Normalbetrieb nicht auf.

***Complete Mediation*:**

Zugriffsanfragen eines Subjekts auf ein Objekt werden *jedes Mal* vollständig auf ihre Zulässigkeit hin überprüft.

◆ Bemerkung

Bei modernen Zero-Trust-Ansätzen wird konzeptionell jeder Zugriff – unabhängig vom Netzwerkstandort – erneut authentifiziert und autorisiert. Dies ist grundsätzlich eine Anwendung des *Principle of Complete Mediation*. Zu unterscheiden sind jedoch Ansätze, die das Ergebnis einer Autorisierungsentscheidung mit fester Ablaufzeit zwischenspeichern (z. B. Authorization-Tokens), und solche, bei denen Rechteänderungen aktiv an alle Durchsetzungspunkte übermittelt werden. Im ersten Fall bleiben entzogene Rechte bis zum Ablauf gültig – das Prinzip wird hier zugunsten von Performance und Skalierbarkeit nur eingeschränkt umgesetzt.

Dieses Prinzip wurde 2012 von Smith[Smith2012] für obsolet erklärt; wie sich jedoch zeigt (Zero-Trust-Ansätze), sind grundlegende Sicherheitsprinzipien von Dauer.

Bei der Entwicklung einer Serveranwendung besagt das *Principle of Complete Mediation* somit, dass bei jeder Anfrage zu überprüfen ist, ob der Nutzer die entsprechenden Rechte besitzt.

***Open Design*:**

Das Prinzip besagt, dass die Sicherheit eines Mechanismus nicht von der Geheimhaltung seines Designs oder seiner Implementierung abhängen sollte.

(Vgl. Kerckhoffs-Prinzip: Die Sicherheit eines Kryptosystems sollte nur von der Geheimhaltung des Schlüssels abhängen.)

***Separation of Privilege*:**

Ein Sicherheitsmechanismus sollte – wenn möglich – nur dann Zugriff gewähren bzw. eine Ausführung erlauben, wenn **mehrere unabhängige Bedingungen zutreffen**. D. h. sollte eine Bedingung kompromittiert sein (z. B. ein abgegriffenes Passwort), dann hat der Angreifer dennoch keinen Zugriff.

Dieses Prinzip wird dann eingehalten, wenn ein Angreifer, der eine Bedingung überwunden hat, nicht die Rechte erhält, um Zugriff auf das System zu erlangen. Zum Beispiel ist es für Geldüberweisungen notwendig, diese auf zwei verschiedene Arten zu autorisieren. Im Allgemeinen ist 2-Faktor-Authentifizierung eine Anwendung dieses Prinzips.

▲ Achtung!

Privilege Separation (für Programme) sollte nicht mit dem hier beschriebenen Prinzip verwechselt werden. *Privilege Separation* liegt zum Beispiel dann vor, wenn ein Programm in zwei Teile aufgeteilt ist und ein Teil – zum Beispiel zum Zugriff auf Betriebssystemressourcen wie Sockets oder bestimmte Dateien – höhere Rechte benötigt als der Rest des Programms. In diesem Fall erfolgt dann der Austausch zwischen den beiden Teilen über eine wohldefinierte, minimale Schnittstelle, die genau festlegt, was der unprivilegierte Teil vom privilegierten Teil anfordern darf. Somit sind die Rechte des unprivilegierten Teils auf das notwendige Minimum beschränkt.

***Least Privilege* (auch bekannt als *Principle of Least Authority*):**

Jedes Programm und jeder Benutzer sollte nur die für seine Aufgabe unbedingt notwendigen Rechte besitzen.

Beispiel: Rechte eines Webservers

D. h. ein Nutzer (zum Beispiel „der Nutzer“, unter dem der Webserver läuft) sollte nach dem Einrichten keine Rechte haben, existierende Dateien, die für den Systembetrieb relevant sind, zu manipulieren, oder solche die sicherheitskritische Informationen beinhalten, zu lesen. Diese Rechte müssen – wenn begründet benötigt – erst explizit gewährt werden.

Beispiel: Eingeschränkte Rechte zum Öffnen von Ports

Ein normaler Nutzer darf auf einem Linux-basierten System keine privilegierten Ports öffnen.

***Least Common Mechanism*:**

Die Sicherheitsmechanismen sollten über Nutzer (insbesondere aber auch Programme, die andere Programme nutzen) hinweg möglichst wenige gemeinsame (Laufzeit-)Abhängigkeiten haben.

(~  *Grundsatz des kleinsten gemeinsamen Mechanismus*)

Beispiel

Das Prinzip besagt zum Beispiel, dass die Mechanismen, die von mehreren Benutzern verwendet werden oder von denen mehrere Nutzer abhängen, minimiert werden sollten.

Das Prinzip kann/sollte auf ganz verschiedenen Ebenen angewendet werden:

- Z. B. sollten keine gemeinsamen Speicherbereiche verwendet werden, in denen möglicherweise sicherheitsrelevantes Material vorgehalten wird; es ist deswegen z. B. sinnvoll – wenn möglich – auf Implementierungen im Kernel zu verzichten und stattdessen auf User-Space-Implementierungen zu setzen.

  TCP-Connection–Hijacking-Angriffe werden bzw. wurden z. B. durch die direkte Implementierung des TCP-Stacks im Linux-Kernel ermöglicht.
- Z. B. wird derselbe (physikalische) Speicher von mehreren Programmen (von verschiedenen Nutzern) genutzt, so gab es in der Vergangenheit mehrere erfolgreiche Angriffe, die es erlaubt haben, aus dem Speicher Information auszulesen, der logisch anderen Programmen zugeordnet war – ggf. sogar über die Grenzen von virtuellen Maschinen hinweg. (Rowhammer, Meltdown/Spectre)

***Psychological Acceptability*:**

Die Sicherheitsmechanismen sollten einfach zu verstehen und zu benutzen sein.

Beispiel: Komplexe Passwortrichtlinien

Komplexe Passwortrichtlinien erweisen sich in der Praxis häufig als kontraproduktiv, da Menschen dann auf vorhersagbare Muster und häufig verwendete Konstruktionsschemata zurückgreifen.

2. Weiterführende (Sicherheits-)Prinzipien

Ausgewählte ergänzende Prinzipien mit Sicherheitsbezug

***Isolation*:** Die Sicherheitsmechanismen sollten so entworfen sein, dass Fehler in einem Teil des Systems nicht die Sicherheit des gesamten Systems gefährden; d. h. die einzelnen Komponenten sollten möglichst unabhängig voneinander sein und nur über wohldefinierte Schnittstellen miteinander kommunizieren und entsprechende Sicherheitsüberprüfungen durchführen.

Beispiel

1. Virtuelle Maschinen können genutzt werden, um Anwendungen in einer isolierten Umgebung auszuführen. Einem Angreifer, der eine Anwendung kompromittiert hat, ist es somit deutlich erschwert, auf andere Anwendungen oder das Betriebssystem zuzugreifen.
2. Typischerweise kommuniziert zum Beispiel ein Basebandchip (LTE, 5G, ...) mit dem Betriebssystem über eine minimale Schnittstelle, über die nur Nachrichten übermittelt werden können, die leicht auf ihre Korrektheit überprüft werden können. Insbesondere sollte kein direkter Zugriff auf den Speicher des Betriebssystems erfolgen.

Einen Angreifer ist es somit ggf. möglich den Basebandchip anzugreifen und ggf. zu kompromittieren, aber er kann nicht direkt auf das Betriebssystem zugreifen und Nachrichten, die bereits auf Betriebssystem oder Anwendungsebene verschlüsselt werden, sind weiterhin sicher.

Modularity: Die Sicherheitsmechanismen sollten so entworfen sein, dass sie unabhängig voneinander implementiert und geprüft werden können.

Hier gibt es Überschneidungen bzw. Anknüpfungspunkte mit dem *Principle of Economy of Mechanism*.

Layering: Die Sicherheitsmechanismen sollten in Schichten organisiert sein.
(Heutzutage spricht man auch von *Defense in Depth* – versagt eine Schicht, greift die nächste.)

Beispiel für ein Schutzsystem für Netzwerke, das mehrere Schichten verwendet:

- einfache (und effiziente) zustandslose Paketfilter auf unterster Ebene
- zustandsbehaftete Paketfilter
- Application-Level-Gateway

Principle of Least Astonishment:
Die Sicherheitsmechanismen sollten so entworfen sein, dass sie keine Überraschungen für die Benutzer bereithalten.

✎ Übung

2.1. *Principle of Open Design*

Benennen Sie ein historisches Verschlüsselungsverfahren, das gegen das *Principle of Open Design* verstoßen hat.

2.2. PINs als Sicherheitsmechanismus

Stellen Sie sich vor, dass Sie als PIN (z. B. für ein Tablet) folgende Zahl verwenden wollen, diese aber abgelehnt wird (Leerzeichen dienen nur der Lesbarkeit):

```
3671 1197 4769
```

Während als PIN das folgende Passwort akzeptiert wird:

```
1364 7964 1364
```

Wie bewerten Sie dies?

※ Hinweis

Schauen Sie sich ggf. ein PIN-Pad an.

✎ Übung

2.3. Browser

Der Chrome-Browser (zum Beispiel) unterstützt die so genannte Isolierung von besuchten Websites. Bei dieser werden Seiten von verschiedenen Websites in unterschiedliche Prozesse aufgeteilt.

Welches Prinzip bzw. welche Prinzipien wird/werden hier umgesetzt?

2.4. Post-Quanten-Kryptografie

Algorithmen und Signaturverfahren, insbesondere solche, die auch im Zeitalter der Quantencomputer noch sicher sein sollen (ML-KEM, ML-DSA und SLH-DSA), wurden bereits im Rahmen von offenen Wettbewerben entwickelt bzw. ausgesucht. Wie bewerten Sie dieses Vorgehen?

✎ Übung

2.5. Rechte von im Hintergrund laufenden Prozessen auf Servern

Es ist üblich, dass für Prozesse, die auf Servern im Hintergrund laufen, eigene Nutzerkonten eingerichtet werden.

- Warum ist dies so?
- Welche Rechte sollten diese „Nutzer“ bekommen?
- Was sollte weiterhin beachtet werden?

Literaturverzeichnis

[Smith2012] R. E. Smith, "A Contemporary Look at Saltzer and Schroeder's 1975 Design Principles", in IEEE Security & Privacy, vol. 10, no. 6, pp. 20-25, Nov.-Dec. 2012, doi: 10.1109/MSP.2012.85

[SaltzerSchroeder75] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," in Proceedings of the IEEE, vol. 63, no. 9, pp. 1278-1308, Sept. 1975, doi: 10.1109/PROC.1975.9939. (Online: Saltzer, Jerome H. and Schroeder, Michael D., "The Protection of Information in Computer Systems", April 1975, https://www.cs.virginia.edu/~evans/cs551/saltzer/)

[Saltzer73] Saltzer, Jerome H., "Protection and control of information sharing in multics", in ACM SIGOPS Oper. Syst. Rev., vol. 7, no. 4, p. 119, Oct. 1973 doi: 10.1145/957195.808059

[Saltzer74] Saltzer, Jerome H., "Protection and the control of information sharing in multics", in Commun. ACM, vol. 17, no. 7, pp. 388 - 402, July 1974 doi: 10.1145/361011.361067